Predicting the Direction of AAPL Daily Returns with a Feed-Forward Neural Network

Python 3.11.5 / scikit-learn 1.7.2 | Data: Yahoo Finance, adjusted daily close, 2015-2024

Summary. A two-hidden-layer neural network using ten lagged returns does not predict the next-day direction of AAPL. Validation balanced accuracy is about 0.50 for every optimizer and architecture, and the final test balanced accuracy is 0.485, below the no-skill level of 0.5. The network trading strategy underperforms buy and hold both before and after transaction costs.

1. Data and experimental design

Stock. Apple Inc. (AAPL): one of the most liquid stocks (tight spreads, realistic low trading costs) with a long, clean price history. Prices are split- and dividend-adjusted.

Definitions. Daily return $r_t = P_t / P_{t-1} - 1$. Inputs $x_t = (r_{t-1}, ..., r_{t-10})$. Target $y_t = 1$ if $r_t > 0$ (Up), else 0 (Down). All inputs are known at the previous close.

	Period	Observations	Up / Down
Full sample	2015-01-20 to 2024-12-31	2,505	53.3% / 46.7%
Model-fitting period (80%)	2015-01-20 to 2023-01-03	2,004	52.5% / 47.5%
Final test period (20%)	2023-01-04 to 2024-12-31	501	56.3% / 43.7%

Observations stay in time order and are never shuffled. Model choices use 5-fold expanding-window time-series cross-validation inside the fitting period. Each fold trains on all earlier data and validates on the next block of about 334 days. Inputs are standardised inside each training fold. The test period is used only once. **Metric:** balanced accuracy = (hit rate on Up days + hit rate on Down days) / 2. Any constant forecast scores 0.5, so 0.5 is the no-skill benchmark.

2. Neural-network model

scikit-learn *MLPClassifier*: inputs (10) -> hidden layers with ReLU activation -> one sigmoid output $p_t = P(y_t = 1 | x_t)$; prediction Up if $p_t > 0.5$. The objective is binary cross-entropy (log loss) with a small L2 penalty (alpha = 10^{-4}). The baseline has 32 and 16 hidden neurons. All optimizers are available in scikit-learn: **SGD** uses mini-batches of 200 with momentum 0.9 and learning rate 0.01; **Adam** uses learning rate 0.001; **L-BFGS** is a full-batch quasi-Newton method. Training stops after 300 iterations, or earlier if the loss improves by less than 10^{-4} for 10 consecutive iterations. For SGD and Adam an iteration is one epoch (a full pass over the data); for L-BFGS it is one quasi-Newton step. Random seed: 42.

3. Optimizer comparison (baseline 32-16 network)

Model	Mean train bal. acc.	Mean val. bal. acc. (sd)	Time (s)	Mean iterations
sgd	0.728	0.494 (0.025)	1.11	255
adam (selected)	0.800	0.504 (0.014)	1.37	295
lbfgs	0.995	0.488 (0.020)	0.58	193

All three optimizers fit the training data well above chance, and L-BFGS almost memorises it (0.99). Yet no optimizer beats 0.5 on validation data by more than about one fold standard deviation. L-BFGS is the fastest on this small data set but overfits the most. SGD and Adam usually stop at the 300-epoch cap. **Adam** has the highest mean validation score and is selected, but the gap between optimizers is within noise.

4. Architecture comparison (Adam)

Model	Mean train bal. acc.	Mean val. bal. acc. (sd)	Time (s)	Mean iterations
32-16 (baseline)	0.800	0.504 (0.014)	1.38	295
16-8 (selected)	0.657	0.506 (0.016)	0.99	270
64-32	0.896	0.502 (0.011)	1.84	277

64-32-16	0.953	0.496 (0.013)	2.30	286

Larger networks fit the training data more closely (0.66 for 16-8 up to 0.95 for 64-32-16), but validation accuracy does not improve and is lowest for the deepest network. The smallest network, **16-8**, has the highest validation score (0.506) and is selected. All differences are within about one standard deviation of no skill.

5. Convergence analysis (SGD and Adam)

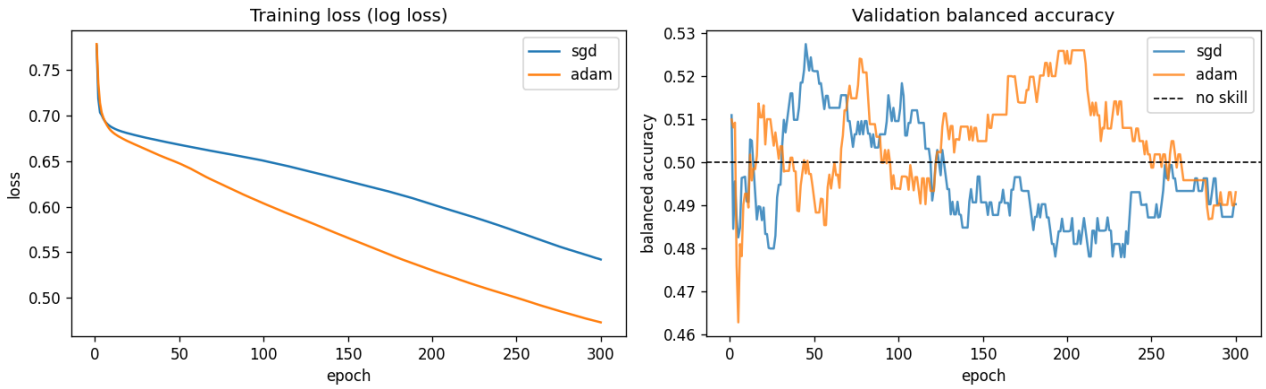


Figure 1. Baseline 32-16 network trained epoch by epoch on fold 5 (train 2015-2021, validate 2021-2023).

The training loss falls steadily for both methods; Adam falls faster (0.78 to 0.47, compared with 0.78 to 0.54 for SGD). Validation balanced accuracy fluctuates around 0.50 throughout. Its peaks of about 0.53 (SGD at epoch 45, Adam at epoch 203) do not persist, and both end near 0.49. This is overfitting: lower training loss reflects noise memorised in past returns, not a repeatable signal.

6. Final evaluation on the untouched test period (2023-2024)

	NN 16-8, Adam	Always Up
Accuracy	0.525	0.563
Balanced accuracy	0.485	0.500
Share of days predicted Up	81.4%	100%

Confusion matrix	Predicted Down	Predicted Up	Hit rate
Actual Down	37	182	16.9%
Actual Up	56	226	80.1%

The network predicts Up on 81% of days. It catches 80% of Up days but only 17% of Down days, so its balanced accuracy (0.485) is *below* 0.5, and its accuracy (0.525) is lower than the naive Always Up rule (0.563). The standard error of balanced accuracy on 501 days is about 0.022, so the result is statistically indistinguishable from no skill. **The network gives no meaningful out-of-sample improvement.** This is consistent with weak-form market efficiency: past returns alone carry no usable information about the next day's direction.

7. Investment strategy

Buy and hold: $R_t = r_t$. **NN strategy:** $s_t = 1$ if $p_t > 0.5$, else 0 (cash); $R_t = s_t\, r_t - c\,|s_t - s_{t-1}|$, with c = 0 (before costs) or c = 10 bp per buy or sell (after costs). Wealth W_T = product of $(1 + R_t)$.

Test period 2023-2024	Final wealth	Ann. return	Ann. vol.	Sharpe	Max DD	In market	Trades
Buy & hold	2.02	42.5%	21.3%	1.77	-16.6%	100%	1
NN (no costs)	1.70	30.7%	19.8%	1.45	-14.8%	81%	147
NN (10 bp costs)	1.47	21.4%	19.8%	1.08	-17.7%	81%	147

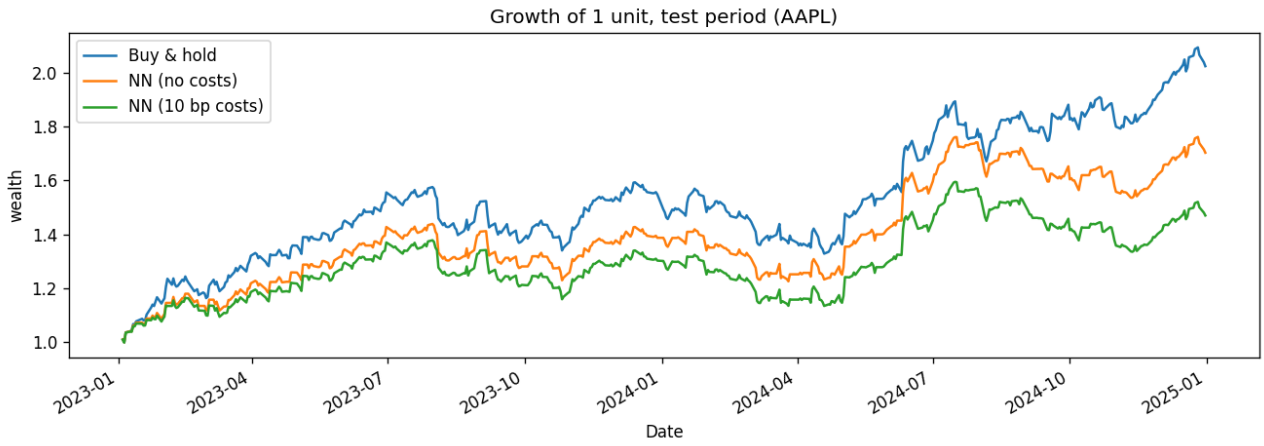


Figure 2. Growth of one monetary unit over the test period.

Buy and hold turns 1 into 2.02 (Sharpe 1.77). The NN strategy holds the stock on 81% of days. Before costs it reaches 1.70 (Sharpe 1.45). Its slightly lower volatility and drawdown come from holding the stock less often, not from successful timing. It trades 147 times, and 10 bp per trade reduces wealth to 1.47 (Sharpe 1.08) and makes its drawdown worse than buy and hold. **Buy and hold dominates the NN strategy both before and after transaction costs.**

8. Conclusion and limitations

A feed-forward network on ten lagged returns learns the training data but not a signal that holds out of sample. The conclusion does not depend on the choice of optimizer or architecture. These results come from a single stock, a single random seed and a single test window, and the exact numbers vary slightly between runs. The no-skill conclusion is stable.

Reproducibility

Software: Python 3.11.5, numpy 2.3.4, pandas 2.3.3, scikit-learn 1.7.2, matplotlib 3.9.2, yfinance 0.2.59, Jupyter.

Files: *nn_stock_direction.ipynb* (all source code); *data/AAPL.csv* (cached prices); *results/*.csv* (all tables); *figures/*.png* (all figures); *requirements.txt*; *README.md*.

To reproduce: `pip install -r requirements.txt`, then `jupyter nbconvert --to notebook --execute --inplace nn_stock_direction.ipynb` (or Run All in Jupyter). The run takes about 20 seconds. All settings are in the first code cell.